

Benchmarking on-demand mobility algorithms using a new remote dispatching interface for MATSim

Sebastian Hörnl^{*a}

^aInstitut de Recherche Technologique SystemX, Palaiseau, France

Submitted for presentation at the MATSim User Meeting 2026 (MUM 2026)

28 September 2026, Paris, France

Abstract

This paper presents a flexible Remote Dispatching Interface (RDI) for MATSim that enables external on-demand mobility dispatching algorithms to interact with MATSim/DRT, including algorithms implemented in languages other than Java. The interface uses ZeroMQ and JSON to exchange vehicle, request, and assignment information between MATSim and external dispatchers through a simple client-server architecture. Its capabilities are demonstrated through five dispatching approaches of increasing complexity, including random assignment, bipartite matching, roaming, insertion, and Q-learning. Benchmarking on a configurable Paris network scenario compares the algorithms in terms of request rejection rates and passenger waiting times across different demand and fleet-size settings. The results demonstrate the practical applicability of the RDI as a flexible framework for developing, testing, and benchmarking alternative mobility-on-demand dispatching methods within MATSim.

1 Introduction

On-demand mobility systems are a major use case for agent-based transport simulations like MATSim (Horni et al., 2016). In such systems, a fleet of vehicles is centrally controlled to pick up and drop off customers, which send their transportation requests, in an optimal way. MATSim provides the DVRP / DRT (Maciejewski et al., 2017) components to simulate such services: While the general traveller agents may send requests with an origin and destination locations, the on-demand vehicles are directly managed by a dispatching algorithm. The vehicles receive coherent schedules with instructions to pick up or drop off customers, or relocate within the road network.

The default dispatching algorithm in MATSim / DRT is based on an insertion heuristic (Bischoff et al., 2017). The algorithm maintains a sequence of pickups, dropoffs and movement tasks into which new tasks can be added. As soon as a new request arrives in the system, the algorithm evaluates, for each vehicle, all viable insertion locations for a new pickup and a new dropoff task along the vehicle’s schedule. The insertion (vehicle, pickup index, dropoff index) that generates the lowest *cost* (quantified in added wait or travel time) is retained. It can happen that no feasible solution is found since any insertion would cause delays that exceed the maximum pick-up and dropoff time constraints of the existing and new constraints. In that case, an incoming request can be rejected.

While this default approach has served as a basis for many methodological and policy-relevant literature on on-demand mobility systems (Hörnl et al., 2021; Kaddoura et al., 2020), alternative approaches exist. A commonly used algorithm is the High-Capacity Ride-Pooling (HCRP) approach by Alonso-Mora et al. (2017). The internal functioning is similar to the insertion approach described above, but all vehicles schedules are dynamically reconstructed optimally in every decision step. While the approach is computationally more complex, it is not limited to optimally treating a request only at the time of insertion. An efficient implementation of the HCRP approach in MATSim is available¹. It makes use of the flexible injection system of the MATSim framework to replace the dispatching-relevant components of DRT. However, just as the default insertion algorithm, many components remain hard-coded and the coupling with the Java-based MATSim codebase remains strong.

Following the general idea of replacing the default dispatching component of MATSim, the present paper proposes a flexible remote dispatching interface (RDI) that makes it possible to connect any new dispatching algorithm written in potentially different (and more common) languages than Java to any MATSim simulation. Thereby, we set the basis to test competing methods inside MATSim and allow researchers from domains that are used to other development environments to test their approaches in simulation.

^{*}Corresponding Author: sebastian.horl@irt-systemx.fr

¹<https://github.com/matsim-org/alonso-mora>

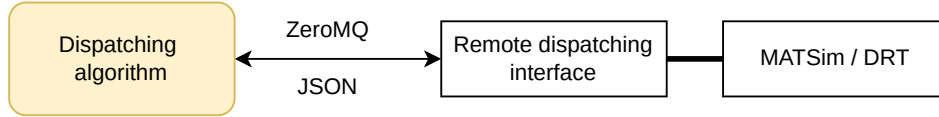


Figure 1: Communication schema

2 Remote dispatching interface

Figure 1 shows the general idea of the dispatching interface. While the MATSim simulation acts as a communication server, a dispatching algorithm can connect as a client. Both entities exchange information in every n th simulation time step. While the simulation sends all relevant *state* information about incoming requests and the fleet, the algorithm sends back detailed instructions for the vehicles. The framework, including various examples, is fully documented and available².

2.1 Implementation

Technically, the RDI is implemented using ZeroMQ as a communication layer. Communication happens in a **REQREP** scheme, which means that each time the client sends a request it is answered by a response from the server. This limited protocol of interaction ensures a predictable and easily maintainable process. ZeroMQ backends exist for many common programming languages such as Java, Python, or Rust.

The exchanged messages are encoded in JSON format for which high-quality serialization libraries exist in many languages. Together, these design choices allow a large user base to connect to MATSim using the RDI.

2.2 Communication protocol

Whenever the RDI is activated, DRT will block before performing any dispatching step. Specifically, if no communication has happened so far, it will wait for an **initialization** message by the algorithm. It is followed by a **start_iteration** message from MATSim. This message is also sent for the start of every following MATSim iteration (usually after one full simulated day).

During one MATSim simulation, the server regularly sends a **state** message containing information on all requests and vehicles that needs to be answered by an **assignment** message, which contains the updated instructions for all vehicles. These instructions can be given at different levels of detail. At the bare minimum, as sequence of pick-up, drop-off or relocation tasks must contain link identifiers. While in this case the required routes through the road network are calculated MATSim-side, it is also possible for the client to provide detailed link sequences.

Between a pair of **state** and **assignment**, the client is allowed to send various optional messages such as **travel_time_request** to inquire about current travel time estimates on a given list of links, or **network_request** to obtain information on all nodes and links.

2.3 Usage

The corresponding RDI repository contains the RDI component for MATSim itself, as well as a separate Java package that includes a simple parametrizable on-demand simulation case based on the road network of Paris. The user can build this package using **maven**, start a MATSim instance and then connect one of four example algorithms implemented in Python to it.

The RDI can be added to any existing MATSim simulation using by adding the Maven package and activating the module for MATSim:

```

ConfigUtils.loadConfig("...", ..., new RemoteDrtConfigGroup());
/// ...
controller.addOverridingModule(new RemoteDrtModule());

```

The RDI is configured *per mode* such that different dispatcher can connect to different on-demand services. For each mode, a specific network port can be configured. If the value is missing, a random

²<https://github.com/sebhoerl/matsim-remote-drt>

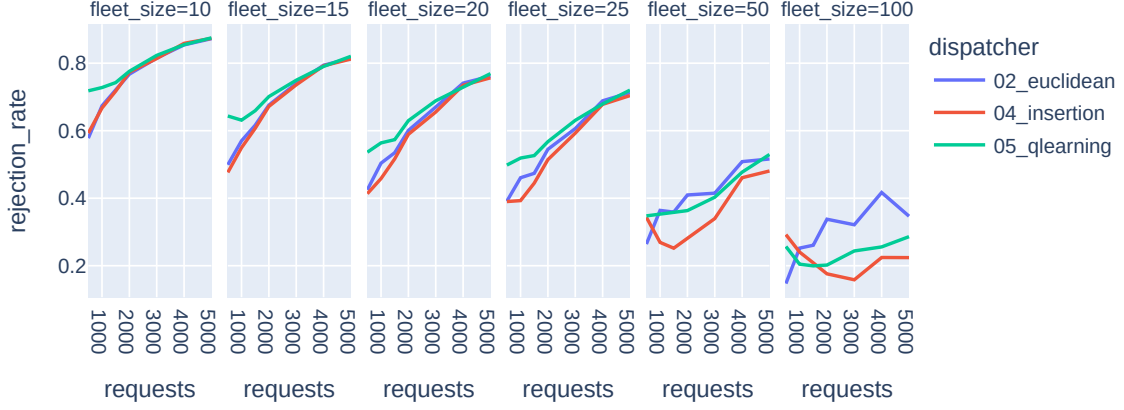


Figure 2: Rejection rates of the benchmarking experiments

port is chosen and written to a temporary file inside the simulation output directory upon start-up. This way, dispatching algorithm can flexibly connect to a running simulation.

3 Benchmarking example

The repository contains five algorithms of increasing complexity showcasing different features of the RDI:

1) The most basic **Random assignment dispatcher** shows how to initiate the connection and then perform random assignments between requests and vehicles. As soon as a vehicle drops off a passenger, a new random request is assigned.

2) The **Bipartite matching dispatcher** showcases how to request the network topology with link and coordinates to put a meaning behind the link identifiers of the requests and vehicles. It maintains a list of waiting requests and unoccupied vehicles and, in every dispatching steps, performs matches them such that the overall Euclidean distance between vehicles and pick-up locations is minimized. The assignments are not binding and can be updated in every decision step.

3) The **Roaming dispatcher** lets vehicles move randomly across the network. As soon as they drive by a customer, the request is assigned to the vehicle. The algorithm first requests the MATSim network and then performs routing client-side using the `networkx` library in Python. This way it showcases how fine-grained routes can be defined.

4) The **Insertion dispatcher** is a simple version of the default MATSim algorithm. It maintains vehicle schedules and finds optimal insertion points for incoming requests. The implementation showcases how requests can be rejected, if necessary, by the RDI.

5) Finally, the **Q-learning dispatcher** represents a more complex use case. It is based on the *bipartite matching dispatcher* but adds a relocation layer for idle vehicles. In particular, it spans a rectangular grid over the road network and learns relocation patterns over the course of *multiple MATSim iterations*. Inspired by experiments in (Chouaki et al., 2022) the number of picked up passengers is treated a reward to learn beneficial relocation flows by time slice.

All algorithms are applied on the provided use case example which generates a configurable number of requests on the road network of Paris. To make the use case interesting, the demand generator specifies demand density sinks and sources that switch their role as the day progresses towards the afternoon. This way, learnable commuting patterns are simulated. Furthermore, the demand changes slightly in every iteration to avoid overfitting in the learning process.

For the benchmark, requests that are not picked up within the specified maximum wait time are automatically rejected server-side to avoid passengers waiting infinitely. Under these conditions, Figure 2 shows the performance of the bipartite matching algorithm, the insertion heuristic, and the Q-learning approach after 100 iterations in terms of rejection rate averaged over 10 seeds and by fleet size and number of generated requests. One can see that generally the rejection rate is strongly dependent on the number of requests and decreases more vehicle supply (boxes). The algorithms perform similarly except when sufficiently many vehicles are available, where, especially, the pooling-enabled insertion algorithm excels.

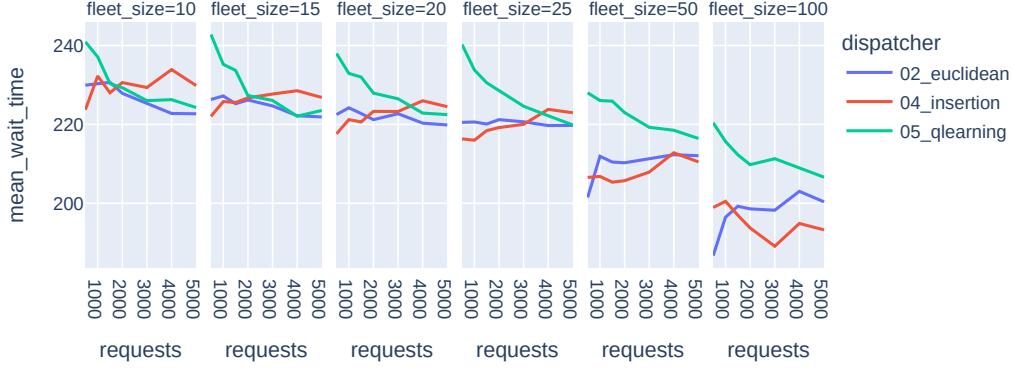


Figure 3: Average wait times of the benchmarking experiments

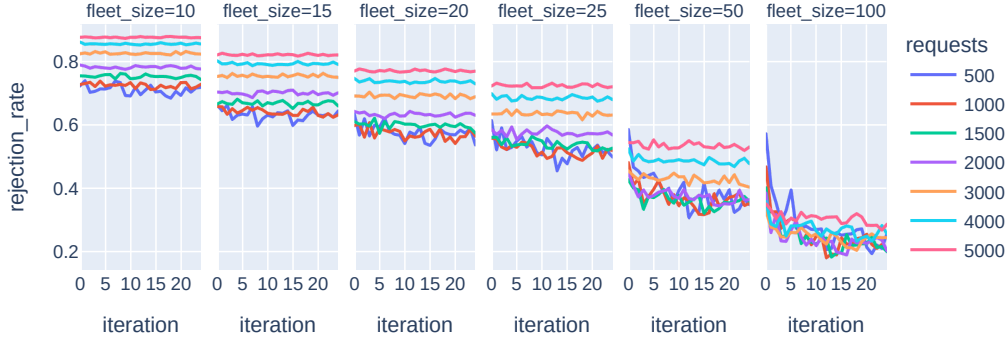


Figure 4: Improvement of average time over multiple iterations using Q-learning

All analyses that can be performed on standard MATSim/DRT runs can be applied to the simulation outputs when using the RDI. Figure 3 shows the performance in terms of wait time (in seconds). One can see especially for high fleet sizes that the Q-learning approach doesn't perform as well as sole bipartite matching (but at the benefit of fewer rejections). The Q-learning approach is an interesting case as it gains information about the demand patterns over the course of one simulation. Figure 4 shows the progression of the average wait time over multiple iterations. Clearly, there is not enough slack with small fleet sizes to perform effective relocation. Only at higher fleet sizes the algorithm visible learns to iteratively reduce wait times.

4 Outlook

The benchmarking examples show that the RDI works and can be used in practice. In particular, the RDI is now used to connect the FleetPy framework (Engelhardt et al., 2026) to MATSim. Furthermore, it has been used in a set-up where a dispatching algorithm is iteratively improved through a code-generating Large Language Model (Hörl et al., 2026).

Several components can still be improved. Especially, providing detailed routes remains a bottleneck as illustrated by the *roaming dispatcher* example. A sensible next step would be to add a routing message that allows to request the server to route a given set of origin-destination pairs MATSim-side, only obtain summary information (travel time, distance) and then reference these routes using a temporary identifier.

Furthermore, it needs to be tested in how far various newer extensions to DRT are compatible with the proposed interface such as prebooking (Hörl et al., 2024) or driver shifts (Zwick et al., 2022).

References

- J. Alonso-Mora, S. Samaranayake, A. Wallar, E. Frazzoli, and D. Rus. On-demand high-capacity ride-sharing via dynamic trip-vehicle assignment. *Proceedings of the National Academy of Sciences*, 114(3): 462–467, Jan. 2017. doi: 10.1073/pnas.1611675114.
- J. Bischoff, M. Maciejewski, and K. Nagel. City-wide shared taxis: A simulation study in Berlin. In *2017 IEEE 20th International Conference on Intelligent Transportation Systems (ITSC)*, pages 275–280, Yokohama, Oct. 2017. IEEE. doi: 10.1109/ITSC.2017.8317926.
- T. Chouaki, S. Hörl, and J. Puchinger. Implementing reinforcement learning for on-demand vehicle rebalancing in MATSim. In *Procedia Computer Science*, volume 201 of *The 13th International Conference on Ambient Systems, Networks and Technologies (ANT 2022)*, pages 134–141, Porto, Portugal, Jan. 2022. doi: 10.1016/j.procs.2022.03.020.
- R. Engelhardt, F. Dandl, A. A. Syed, C. Ding, S. Alvarez-Ossorio Martinez, Y. Zhang, H. Hamdy, J. Brodersen, Z. Chen, and K. Bogenberger. FleetPy: an open source simulator for reproducible research on mobility-on-demand services. *European Transport Research Review*, 18(1):52, July 2026. doi: 10.1186/s12544-026-00823-3.
- A. Horni, K. Nagel, and K. W. Axhausen, editors. *The Multi-Agent Transport Simulation MATSim*. Ubiquity Press, Aug. 2016. doi: 10.5334/baw.
- S. Hörl, F. Becker, and K. W. Axhausen. Simulation of price, customer behaviour and system impact for a cost-covering automated taxi system in Zurich. *Transportation Research Part C: Emerging Technologies*, 123:102974, Feb. 2021. doi: 10.1016/j.trc.2021.102974.
- S. Hörl, T. Chouaki, O. Ludwig, H. Rewald, and S. Axer. Evaluating prebooked on-demand mobility services using MATSim. In *Procedia Computer Science*, volume 238, pages 763–770, Hasselt, Belgium, Jan. 2024. doi: 10.1016/j.procs.2024.06.089.
- S. Hörl, M. Gagnant, and M. Royer. Iterative LLM-based construction of on-demand fleet dispatching algorithms. In *24th International Conference on Practical Applications of Agents and Multi-Agent Systems (PAAMS 2026)*, Naples, Italy, Oct. 2026. URL <https://hal.science/hal-05671885v1>.
- I. Kaddoura, J. Bischoff, and K. Nagel. Towards welfare optimal operation of innovative mobility concepts: External cost pricing in a world of shared autonomous vehicles. *Transportation Research Part A: Policy and Practice*, 136:48–63, June 2020. doi: 10.1016/j.tra.2020.03.032.
- M. Maciejewski, J. Bischoff, S. Hörl, and K. Nagel. Towards a Testbed for Dynamic Vehicle Routing Algorithms. In J. Bajo, Z. Vale, K. Hallenborg, A. P. Rocha, P. Mathieu, P. Pawlewski, E. Del Val, P. Novais, F. Lopes, N. D. Duque Méndez, V. Julián, and J. Holmgren, editors, *Highlights of Practical Applications of Cyber-Physical Multi-Agent Systems*, pages 69–79, Cham, 2017. Springer International Publishing. doi: 10.1007/978-3-319-60285-1_6. ipr.
- F. Zwick, N. Kuehnel, and S. Hörl. Shifts in perspective: Operational aspects in (non-)autonomous ride-pooling simulations. *Transportation Research Part A: Policy and Practice*, 165:300–320, 2022. doi: 10.1016/j.tra.2022.09.001.